Let me get the full Unsloth dequantization script which handles this correctly:Let me check the actual config to understand the FP8 scaling block size:Let me dig into the proper dequantization approach for this model:Let me look at the FP8 → NVFP4 converter used by RedHat for the official NVFP4 checkpoint:Let me look at the proper DeepSeek-V3 reference dequantization code:Let me fetch the actual PyTorch reference implementation:Let me check for Mistral-Small-4 MoE calibration support in llm-compressor:Your diagnosis is correct — the dequantization step is where everything dies, and every downstream layer inherits that corruption. There are actually **two distinct bugs** in the snippet Mistral ships in the model card (and that Unsloth copies). Fix those and AWQ-int4 via llm-compressor works fine on A100-80.

Why tensor * scale_inv produces garbage

Mistral Small 4 stores its FP8 weights in **DeepSeek-V3-style 128×128 block-scaled FP8** (e4m3). That means for a [out, in] weight, weight_scale_inv has shape [ceil(out/128), ceil(in/128)] — one scale per 128×128 tile. The snippet on the HF page:

return (tensor.to(torch.bfloat16) * scale_inv.to(torch.bfloat16)).to(torch.bfloat16)

has two fatal problems:

1. **No block broadcast.** Element-wise multiply of [out, in] against [out/128, in/128] only "works" in the rare case shapes happen to align as 1×1 broadcasts — most of the time it silently misaligns scales to weights, hence the universal garbage you're seeing regardless of attention backend, arch wrapper, or downstream quantizer.
2. **Multiply done in BF16.** Even when shapes align (per-tensor or per-channel scales for MoE expert tensors), doing the multiply in BF16 throws away precision — FP8 e4m3 has a 3-bit mantissa, and you need at least FP32 intermediate to preserve the product.

The reference is DeepSeek's weight_dequant in inference/kernel.py. Here it is as plain PyTorch, which is what you want for an offline conversion:

```
import torch


def fp8_block_dequant(weight_fp8: torch.Tensor,
                      scale_inv: torch.Tensor,
                      block_size: int = 128) -> torch.Tensor:
    """DeepSeek-V3 / Mistral-Small-4 style 128x128 block FP8 -> BF16."""
    # Upcast FP8 to FP32 BEFORE multiply (critical)
```

```
    w = weight_fp8.to(torch.float32)
    out, in_ = w.shape

    # Pad to multiple of 128 (Mistral pads internally; we mirror it)
    pad_out = (block_size - out % block_size) % block_size
    pad_in  = (block_size - in_ % block_size) % block_size
    if pad_out or pad_in:
        w = torch.nn.functional.pad(w, (0, pad_in, 0, pad_out))

    # Expand scales from [out/128, in/128] -> [out_padded, in_padded]
    s = scale_inv.to(torch.float32)
    s = s.repeat_interleave(block_size, dim=0).repeat_interleave(block_size, dim=1)
    s = s[: w.shape[0], : w.shape[1]]

    deq = (w * s)[:out, :in_]            # crop padding
    return deq.to(torch.bfloat16)
```

For the **MoE expert** tensors (gate_up_proj, down_proj, up_proj) the scale layout is different — they're stored as [num_experts, out, in] with a [num_experts, out/128, in/128] scale. Loop over experts and apply the same function per slice. Activation scales (*_activation_scale) are dynamic — drop them, llm-compressor will create its own.

End-to-end llm-compressor AWQ recipe

Two things to get right that are easy to miss for this specific model:

- **Multimodal head.** Mistral3ForConditionalGeneration has a vision_tower (Pixtral encoder) and a multi_modal_projector.patch_merger.merging_layer. There's an open issue (#1652 in llm-compressor) where vLLM rejects the load because the patch-merger Linear ends up packed. You need to add those to ignore.
- **MoE expert calibration.** Standard oneshot only routes through the experts the router picks, so most experts get zero calibration data and quantize to noise. Use

MoECalibrationModule (introduced in 0.9.0) — or, since Mistral Small 4 doesn't have a built-in calibration replacement like GLM-4 does, set aux_loss_alpha / route all experts during forward by patching the MoE module (the pattern in examples/quantizing_moe/).

Install:

```
pip install -U "llmcompressor>=0.10.0" "compressed-tensors>=0.9" \
            "transformers>=4.50" "vllm>=0.7"
```

Then the recipe:

```
from transformers import AutoTokenizer, Mistral3ForConditionalGeneration
from llmcompressor import oneshot
from llmcompressor.modifiers.awq import AWQModifier
from llmcompressor.modifiers.quantization import QuantizationModifier
from datasets import load_dataset

MODEL_ID = "/path/to/Mistral-Small-4-119B-2603-BF16"   # the dequantized model

model = Mistral3ForConditionalGeneration.from_pretrained(
   MODEL_ID, dtype="bfloat16", device_map="auto",
   # disk offload if you don't have ~250GB CPU RAM:
   offload_folder="/scratch/offload", offload_state_dict=True,
)
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)

ignore = [
   "lm_head",
   "re:vision_tower.*",              # Pixtral encoder — keep BF16
   "re:multi_modal_projector.*",     # incl. patch_merger.merging_layer
   "re:.*router.*",                  # MoE gating — keep BF16, very small
```

```
    # Optional: keep the first 2-3 dense decoder layers full-precision
    # (Mistral Small 4 has dense layers up front before the MoE blocks)
    "re:model.language_model.layers.[0-2]\\..*",
    # Bartowski found later-layer down_proj_exps are NaN-sensitive — protect them
    "re:model.language_model.layers.(4[0-9]|5[0-9])\\.feed_forward.experts.\\d+.down_proj",
]

recipe = [
    AWQModifier(
        bits=4,
        symmetric=False,
        group_size=128,
        duo_scaling=True,          # required for per-group strategies
    ),
    QuantizationModifier(
        targets=["Linear"],
        scheme="W4A16_ASYM",
        ignore=ignore,
    ),
]

# Small text-only calibration set — vision is ignored anyway
ds = load_dataset("HuggingFaceH4/ultrachat_200k", split="train_sft[:512]")
ds = ds.map(lambda x: {"text": tokenizer.apply_chat_template(
    x["messages"], tokenize=False)})
```

```
oneshot(
    model=model,
    dataset=ds,
    recipe=recipe,
    max_seq_length=2048,
    num_calibration_samples=512,
    output_dir="Mistral-Small-4-119B-2603-AWQ-INT4",
)
```

Then in vLLM (A100, so explicitly Marlin — **not** FP4):

```
vllm serve ./Mistral-Small-4-119B-2603-AWQ-INT4 \
  --quantization awq_marlin \
  --max-model-len 16384 \
  --gpu-memory-utilization 0.92 \
  --tool-call-parser mistral --enable-auto-tool-choice \
  --reasoning-parser mistral
```

Reality check on A100-80

- **Disk/RAM during conversion**: dequantized BF16 is ~240 GB on disk and roughly the same in CPU RAM (with offload) during AWQ calibration. The actual GPU forward pass at calibration time will fit on one A100, but you need a host with at least ~300 GB free RAM+swap+SSD or you'll OOM during oneshot even with offload_state_dict=True.
- **Final size**: ~60 GB for INT4 weights + ~2 GB BF16 vision + projector + router + sensitive layers ≈ **63–65 GB on device**. That leaves ~15 GB for KV cache + activations, which is why I capped --max-model-len at 16k above. Don't try 262k on one card.
- **If you don't have the host memory** to do the dequant + calibration: rent an H100/H200 node with 1TB RAM for an hour, run the recipe there, then serve the resulting 60 GB INT4 checkpoint on your A100.

One alternative worth knowing about: llm-compressor 0.9+ has a model_free_ptq pathway that operates directly on safetensors files and was specifically used by Red Hat to quantize Mistral Large 3 — but it's currently **data-free only** (FP8/NVFP4A16), so it won't do AWQ-int4 for you. If

you'd accept W4A16 RTN instead of activation-aware AWQ, that path skips the dequant-and-load-full-model dance entirely, but quality will be noticeably worse on a 4-active-expert MoE.